

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Вычислительной техники

Курсовая работа
по дисциплине "Программирование"
Тема: Обработка текстовой информации

Студент гр. 4311

_____ Ласков С. В.

Преподаватель

_____ Хахаев. И.А

Санкт-Петербург

2024

Введение

Цель работы

Законченное поэтапное решение содержательной задачи (постановка задачи, спецификация, выбор структур данных и разработка алгоритма, программная реализация, тестирование).

Задание (вариант 68)

Задан текст, содержащий произвольное количество строк, в которых отдельные слова могут разделяться одним или несколькими пробелами и знаками пунктуации (перенос слов с одной строки на другую не используется). Сформировать новый текст, который является результатом следующего преобразования исходного текста: упорядочить в каждой строке слова в алфавитном порядке.

Постановка задачи и описание решения

Задача программы заключается в обработке текста, состоящего из произвольного количества строк, в которых слова могут быть разделены одним или несколькими пробелами и знаками пунктуации. Цель состоит в том, чтобы упорядочить слова в каждой строке в алфавитном порядке и сформировать новый текст на основе этих преобразований. Программа должна поддерживать ввод данных как с консоли, так и из файла, а также выводить результат на экран или сохранять его в файл.

Основные требования:

1. Ввод данных: Пользователь должен иметь возможность выбрать способ ввода данных — с консоли или из файла. Это обеспечит гибкость и удобство работы с текстом.
2. Обработка строк: Каждая строка должна быть обработана для удаления лишних пробелов и знаков пунктуации. Важно, чтобы программа могла корректно обрабатывать различные

форматы ввода, включая наличие нескольких пробелов между словами и различные знаки препинания.

3. Сортировка слов: после обработки строки необходимо отсортировать слова в алфавитном порядке. Это позволит пользователю получить структурированный текст, где слова упорядочены по правилам алфавита.
4. Вывод результата: Результат должен быть представлен пользователю как в виде текста на экране, так и в виде сохраненного файла. Это даст возможность пользователю сохранить результаты своей работы для дальнейшего использования.

Решение данной задачи можно разделить на несколько этапов:

1. Обработка пользовательских данных

В основной функции программы пользователю предлагается выбрать способ ввода данных. Это может быть реализовано через простое меню, где пользователь выбирает между вводом с консоли и вводом из файла.

- Ввод с консоли: Если пользователь выбирает этот вариант, программа запрашивает у него количество строк, которые он собирается ввести, а затем последовательно считывает каждую строку. Такой подход позволяет пользователю быстро вводить данные вручную.
- Ввод из файла: Если выбран ввод из файла, программа запрашивает имя файла, открывает его для чтения и считывает строки. В этом случае программа должна учитывать возможные ошибки при открытии файла (например, если файл не существует), чтобы избежать сбоев в работе.

2. Обработка и сортировка строк

После получения строк от пользователя программа переходит к их обработке:

- Удаление лишних пробелов и знаков пунктуации: Каждая строка должна быть очищена от лишних пробелов и знаков препинания. Важно обеспечить корректное выделение слов даже в случае сложных комбинаций пробелов и знаков препинания.
- Сортировка слов: после очистки строки она разбивается на отдельные слова, которые затем сортируются в алфавитном порядке. Этот шаг обеспечивает структурированность текста и делает его более читабельным.

3. Формирование нового текста

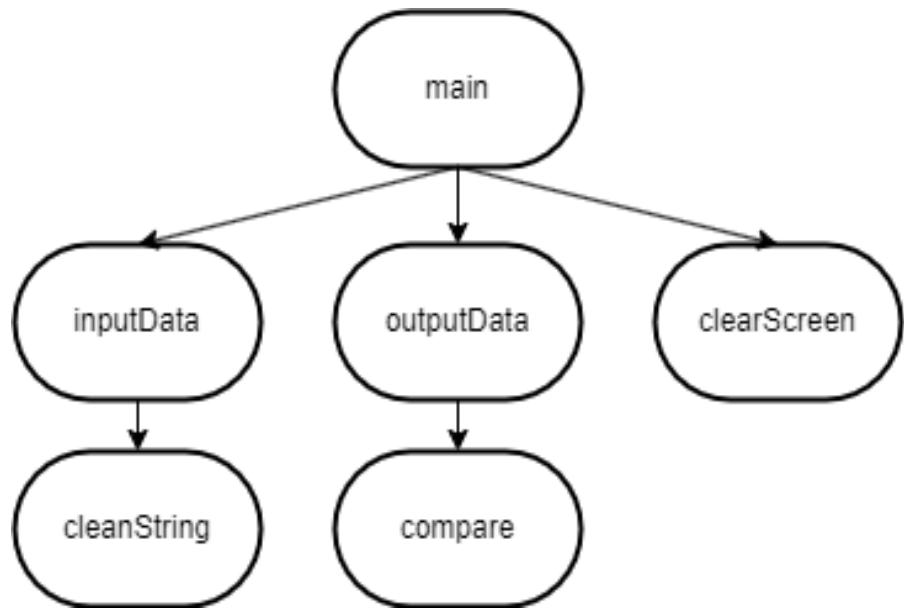
После обработки всех строк программа формирует новый текст, состоящий из отсортированных строк.

4. Вывод результата

На заключительном этапе программа предлагает пользователю выбрать способ вывода результата:

- Вывод на экран: При выборе этого варианта программа отображает как исходный текст, так и новый текст с отсортированными словами. Это позволяет пользователю сразу увидеть результаты обработки.
- Сохранение в файл: Если пользователь выбирает сохранение результата в файл, программа записывает как исходный текст, так и отсортированный текст в файл (например, output.txt). После успешного сохранения программа выводит сообщение о том, что результаты были успешно записаны.

Структура вызовов функций



Описание функций

1. `int compare(const void *a, const void *b)`

Функция сравнения для использования с `qsort`. Сравнивает два указателя на строки и возвращает:

- отрицательное значение, если первая строка меньше второй,
- 0, если строки равны,
- положительное значение, если первая строка больше второй.

2. `void cleanString(char *line)`

Очищает строку от лишних пробелов:

- Удаляет ведущие и завершающие пробелы.
- Убирает лишние пробелы между словами.

3. `void clearScreen()`

Очищает экран консоли:

- Для Windows выполняет команду `cls`.
- Для других систем (Linux, macOS) выполняет команду `clear`.

4. `void inputData(char lines[MAX_LINES][MAX_LENGTH], int *lineCount, int fromFile)`

Вводит данные:

- Если `fromFile == 1`, данные считываются из файла.
- Если `fromFile == 0`, данные вводятся с консоли.
- Строки очищаются от лишних пробелов с помощью функции `cleanString`.

5. `void outputData(char lines[MAX_LINES][MAX_LENGTH], int lineCount, int toFile, int isOriginal)`

Выводит данные:

- Если `toFile == 1`, данные записываются в файл.
- Если `toFile == 0`, данные выводятся на консоль.
- Если `isOriginal == 1`, выводятся оригинальные строки.
- Если `isOriginal == 0`, выводятся отсортированные строки (слова внутри строк сортируются лексикографически).

6. `int main()`

Основная функция, управляющая процессом:

- Меню для выбора ввода (с консоли или из файла).
- Меню для выбора вывода (на консоль или в файл).
- Использует функции `inputData` и `outputData` для обработки ввода и вывода данных.

Описание переменных

Функция `compare()`

№	Название переменной	Тип	Назначение
1	a	const void	указатели на элементы для сравнения (строки).

2	b	const void	указатели на элементы для сравнения (строки).
---	---	------------	---

Функция cleanString()

№	Название переменной	Тип	Назначение
1	line	char	Указатель на строку, которую нужно очистить от лишних пробелов.
2	src	char	Указатель на исходную строку, с которой будет происходить чтение символов.
3	dst	char	Указатель на место в строке line, куда будут копироваться символы после удаления лишних пробелов.

Функция inputData()

№	Название переменной	Тип	Назначение
1	lines[MAX_LINES][MAX_LENGTH]	char	массив строк, в который будут

			сохранены введенные данные.
2	lineCount	int	указатель на целочисленную переменную, в которой будет храниться количество введенных строк.
3	fromFile	int	переменная, которая указывает, откуда брать данные.
4	filename	char	Массив символов для хранения имени файла, с которым будет производиться работа при чтении данных из файла.
5	file	FILE*	Указатель на объект файла, который

			используется для работы с файлом при чтении.
6	i	int	Индекс для циклов, используемых для записи строк в массив lines.
7	done	int	Флаг, который используется для завершения ввода строк с консоли.

Функция outputData()

№	Название переменной	Тип	Назначение
1	lines[MAX_LINES][MAX_LENGTH]	char	массив строк, в который будут сохранены введенные данные.
2	lineCount	int	указатель на целочисленную переменную, в которой будет храниться количество введенных строк.

3	fromFile	int	переменная, которая указывает, откуда брать данные.
4	isOriginal	int	переменная, которая указывает, нужно ли выводить оригинальные строки или отсортированные.
5	filename	char	Массив символов, который используется для хранения имени файла, в который будут записаны данные.
6	file	FILE*	Указатель на файл, в который будет производиться вывод данных, если toFile равно 1.
7	i	int	Индексы для циклов, которые используются для перебора строк и слов в строках.
8	j	int	Индексы для циклов, которые используются для перебора строк и слов в строках.
9	words[MAX_WORDS]	char	Массив указателей на строки (массив строк), в

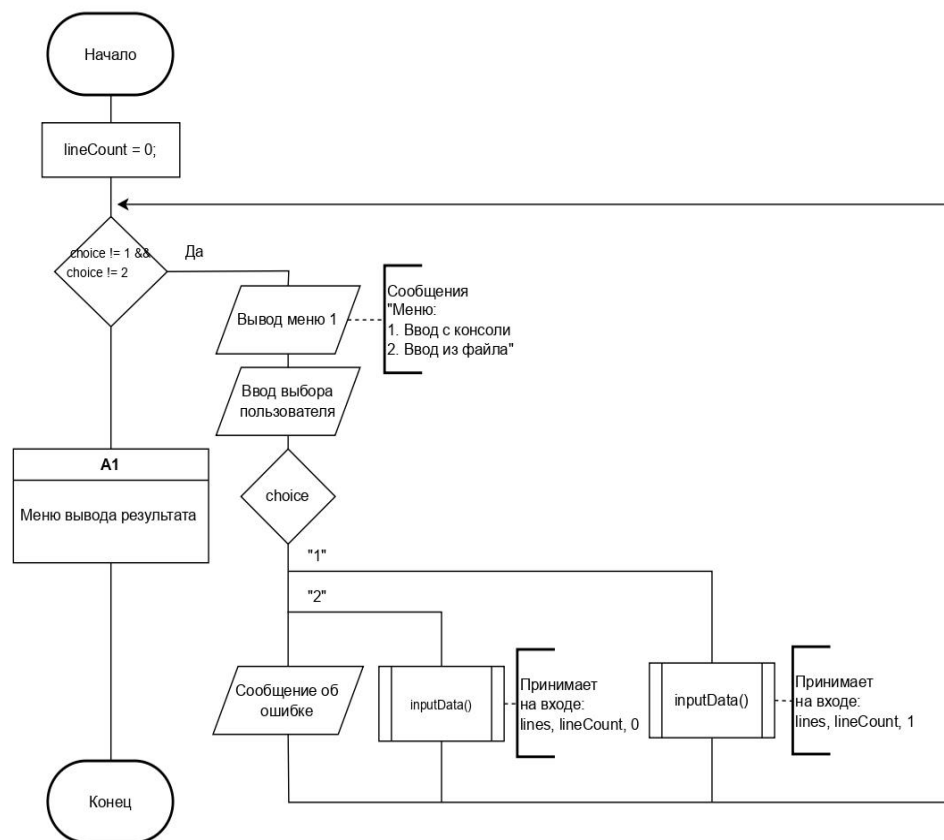
			котором будут храниться слова, извлеченные из строки. Это нужно для сортировки слов в строках.
10	wordCount	int	Целое число, которое хранит количество слов в текущей строке. Используется при работе с массивом words.
11	token	int	Указатель на строку, представляющую собой отдельное слово, извлеченное из текущей строки с помощью функции strtok().

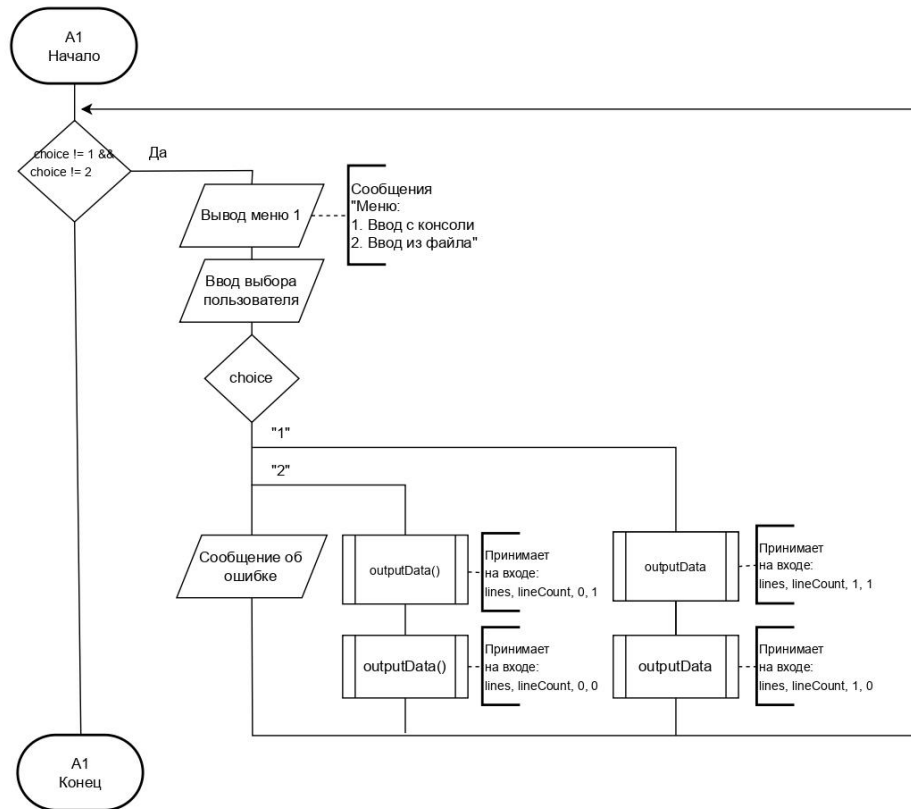
Функция main()

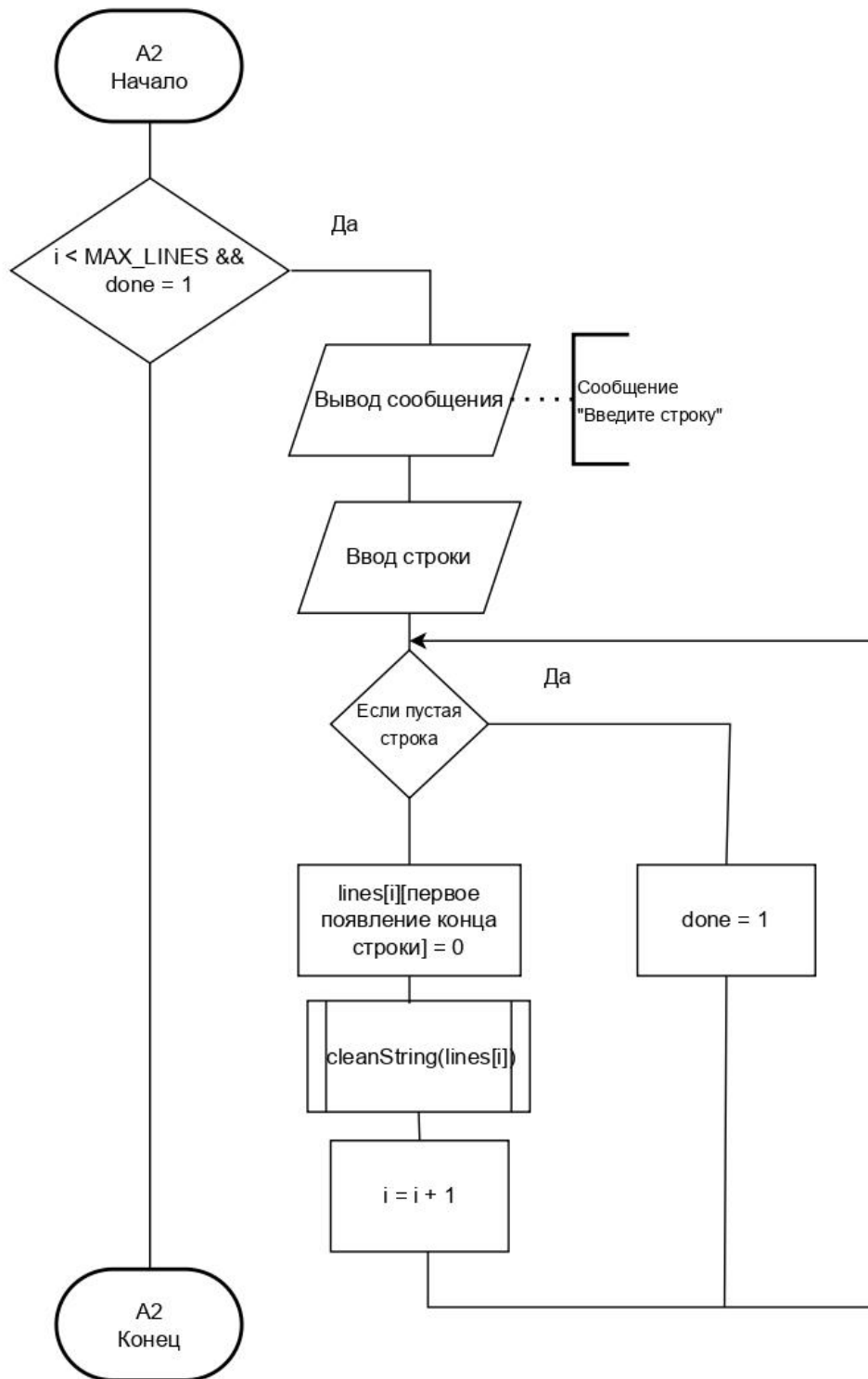
№	Название переменной	Тип	Назначение
1	lines[MAX_LINES][MAX_LENGTH]	char	массив строк для хранения введенных данных.
2	lineCount	int	целое число, которое хранит количество строк в массиве lines,

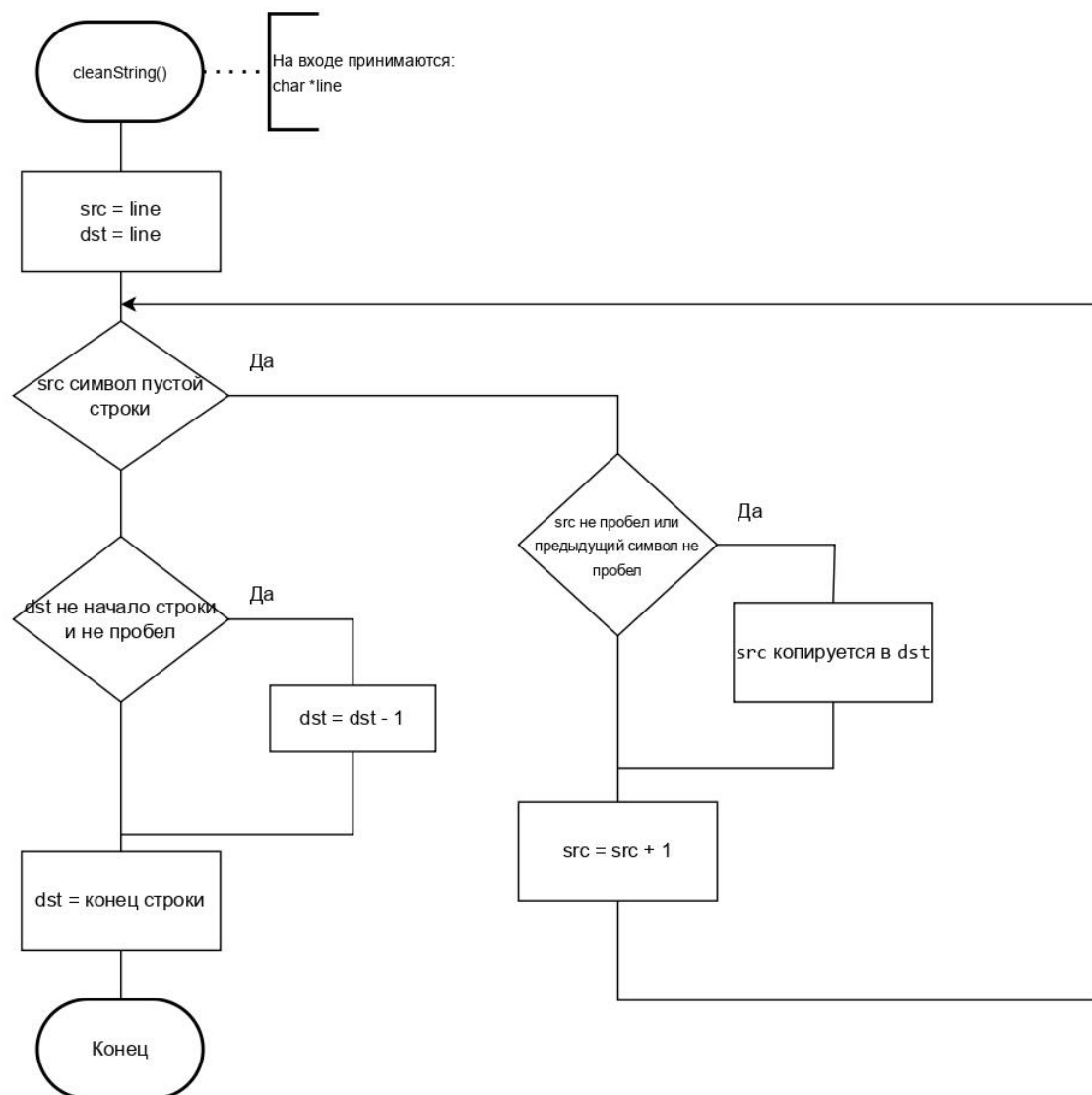
			введенных пользователем.
3	choice	int	целое число, которое используется для хранения выбора пользователя в меню ввода и вывода данных.

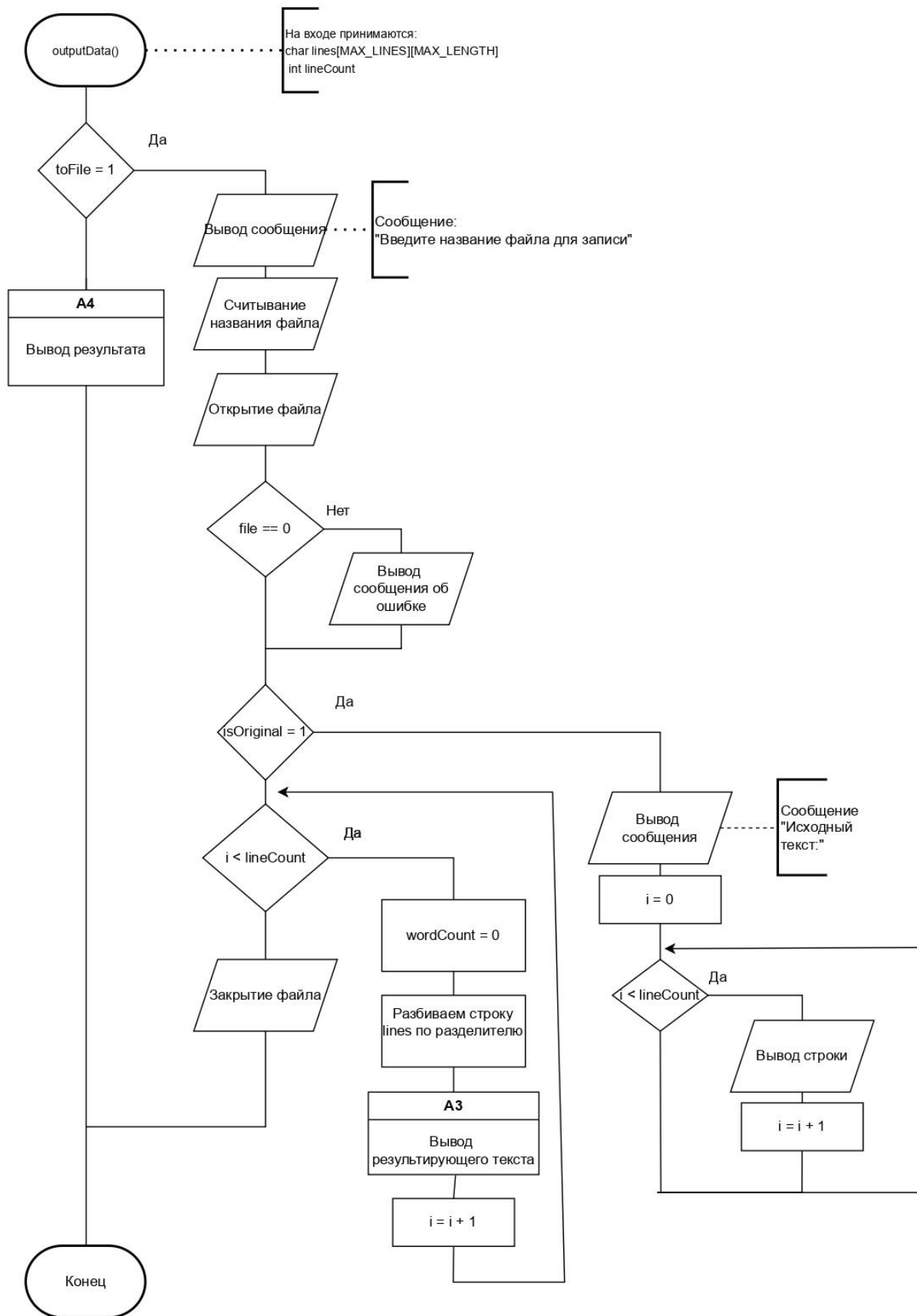
Схема алгоритма

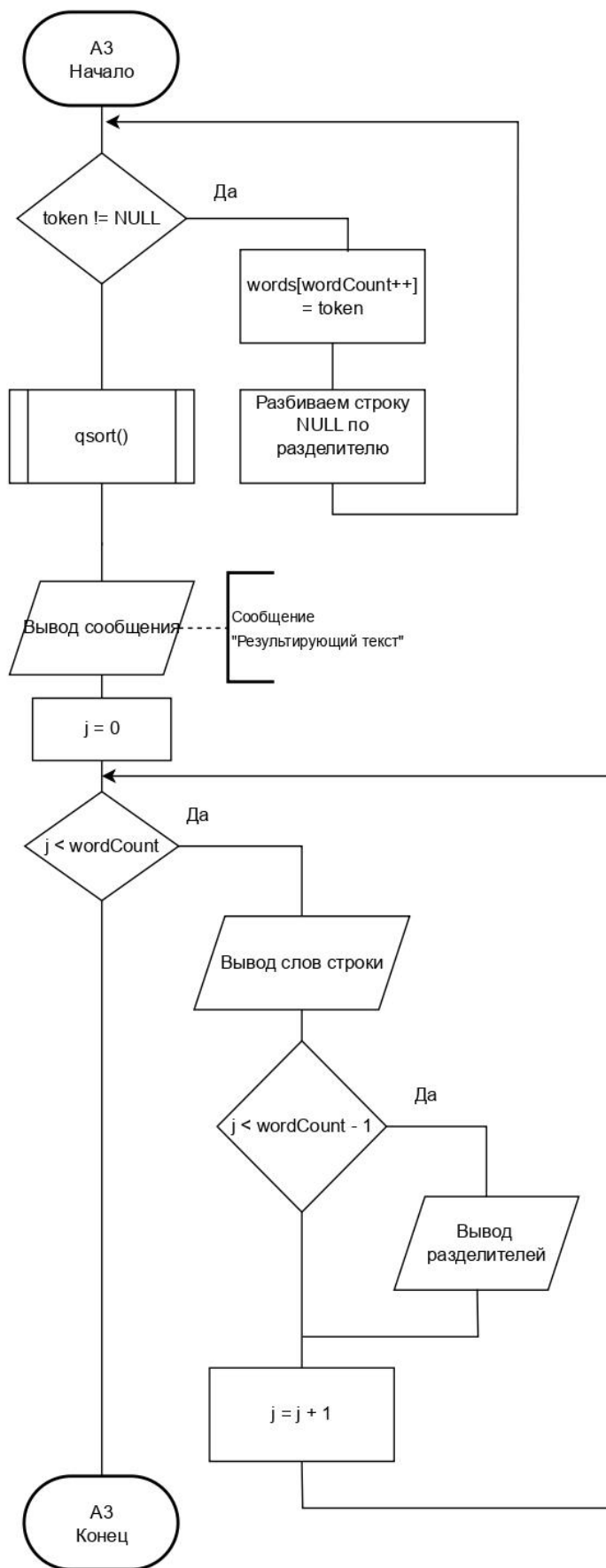


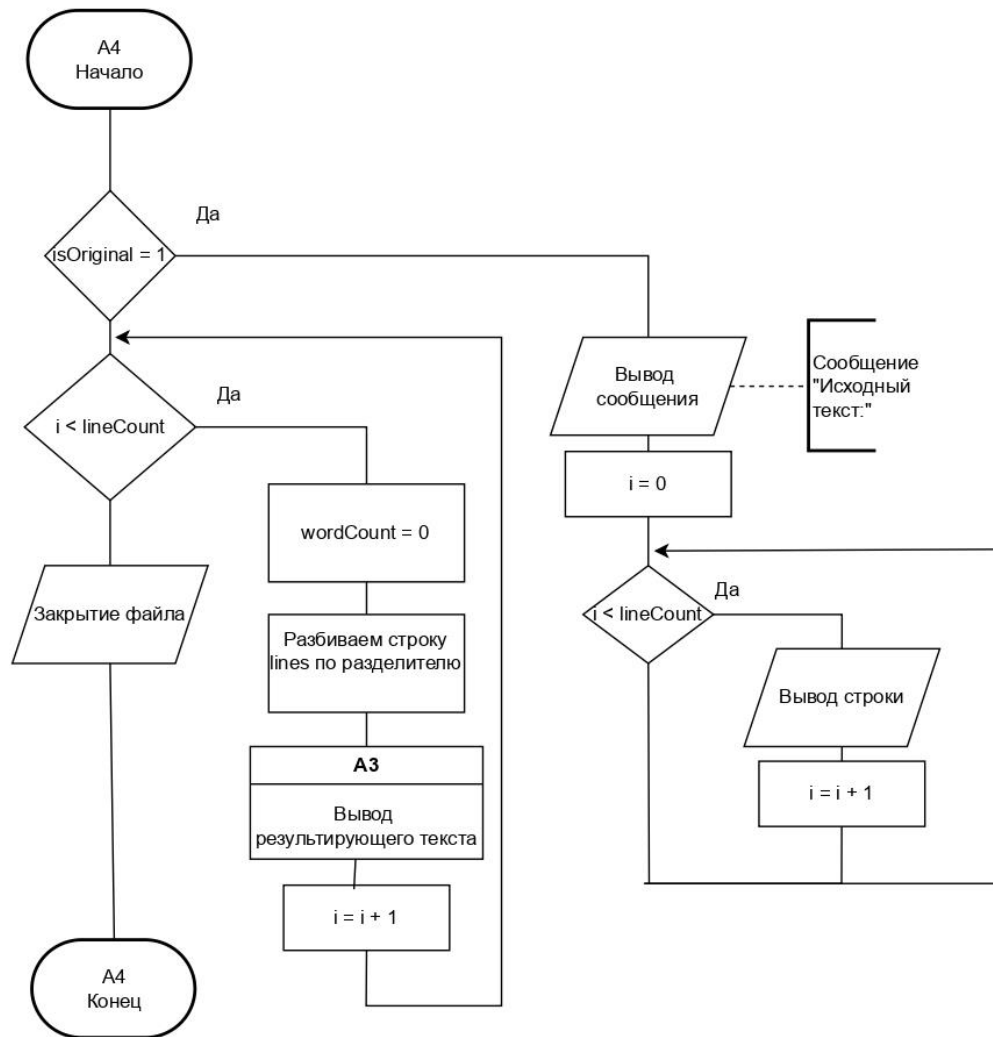


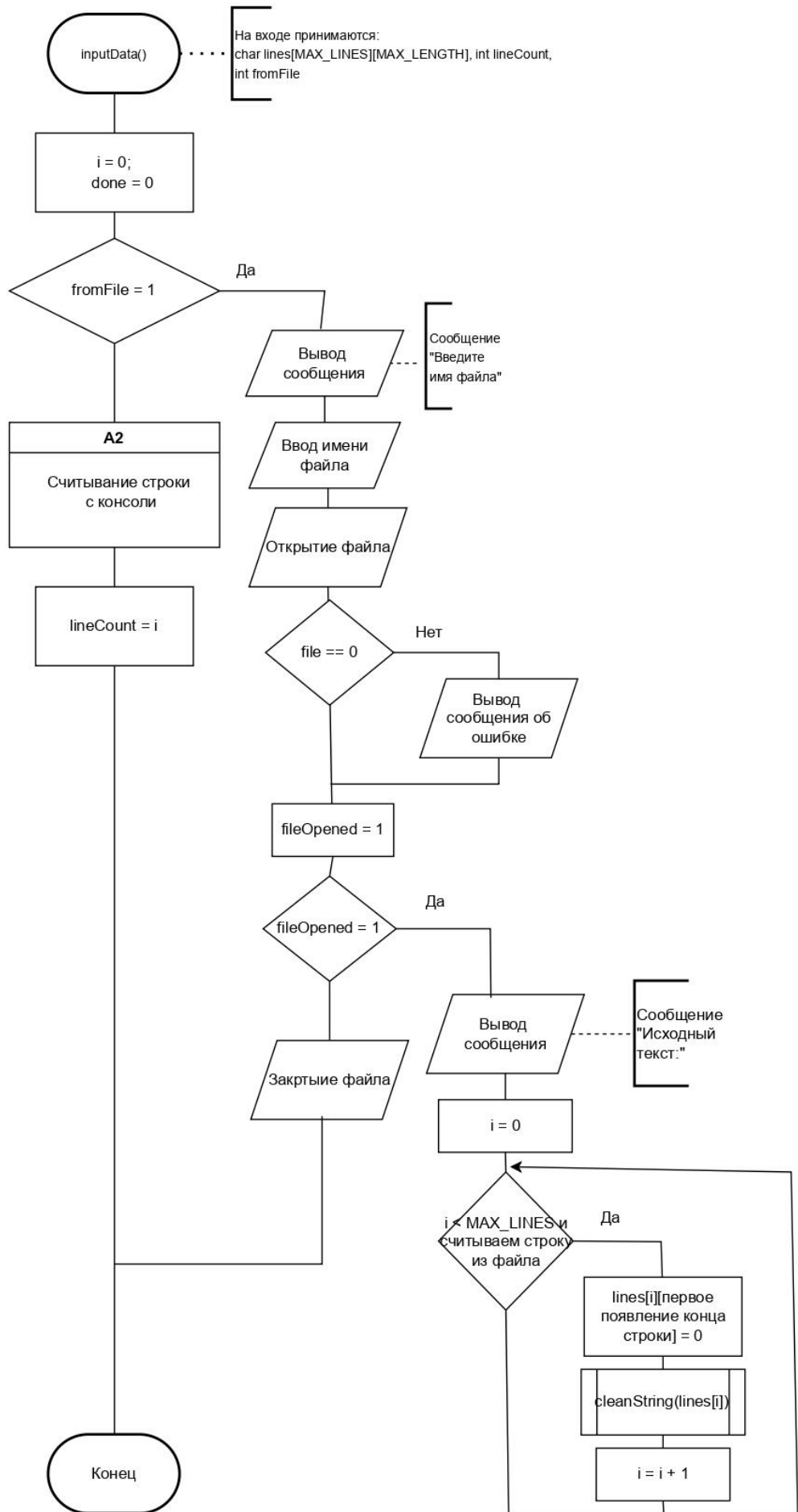












Контрольные примеры

№	Исходные данные	Результаты
1	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 1 Enter a line (or press Enter to finish): This is the first line. Enter a line (or press Enter to finish): This is the second line. Enter a line (or press Enter to finish): This is the third line. Enter a line (or press Enter to finish): (Press Enter) Output Menu: 1. Output data to console 2. Output data to file Choose an option: 1	Original text: This is the first line. This is the second line. This is the third line. Transformed line: This first is line. the Transformed line: This is line. second the Transformed line: This is line. the third
2	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 2 Enter the filename to read: Firstline.txt Output Menu: 1. Output data to console 2. Output data to file Choose an option: 1	Original text: First line Second line Third line Transformed line: Second line. Third line. First line. Transformed line: Second line. Transformed line: Third line.
3	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 1 Enter a line (or press Enter to finish): Hello. Enter a line (or press Enter to finish): Hi. Enter a line (or press Enter to finish): (Press Enter)	Original text: Hello. Hi. Transformed line: Hi. Hello. Transformed line: Hi.

	Output Menu: 1. Output data to console 2. Output data to file Choose an option: 1	
4	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 1 Enter a line (or press Enter to finish): This is the first example. Enter a line (or press Enter to finish): This is the second example. Enter a line (or press Enter to finish): (Press Enter) Output Menu: 1. Output data to console 2. Output data to file Choose an option: 2 Enter the filename to write original text: output.txt Enter the filename to write transformed text: output.txt	Original text: This is the first example. This is the second example. Transformed line: This is the second example. This is the first example. Transformed line: This is the second example.
5	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 3	Invalid choice. Please try again.
6	Input Menu: 1. Input data from console 2. Input data from file Choose an option: 2 Enter the filename to read: non_existent_file.txt	Failed to open the file.

Текст программы

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```

#define MAX_LINES 100
#define MAX_LENGTH 1000
#define MAX_WORDS 100

/* Function prototypes */
int compare(const void *a, const void *b);
void cleanString(char *line);
void clearScreen();
void inputData(char lines[MAX_LINES][MAX_LENGTH], int
*lineCount, int fromFile);
void outputData(char lines[MAX_LINES][MAX_LENGTH], int
lineCount, int toFile, int isOriginal);

/* Function to compare two strings for qsort */
int compare(const void *a, const void *b) {
    const char *str1 = *(const char **)a;
    const char *str2 = *(const char **)b;
    return strcmp(str1, str2);
}

/* Function to clean up a string by removing extra spaces */
void cleanString(char *line) {
    char *src = line, *dst = line;
    while (*src) {
        if (!isspace(*src) || (dst != line && !isspace(*(dst - 1)))) {
            *dst++ = *src;
        }
        src++;
    }
}

```

```

    }
    if (dst != line && isspace(*(dst - 1))) {
        dst--;
    }
    *dst = '\0';
}

```

/* Function to clear the console screen */

```

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

```

/* Function to input data (from console or file) */

```

void inputData(char lines[MAX_LINES][MAX_LENGTH], int
*lineCount, int fromFile) {
    char filename[MAX_LENGTH];
    FILE *file;
    int i;
    int don;
    i = 0;
    done = 0;
    if (fromFile) {
        printf("Enter the filename to read: ");
        scanf("%s", filename);
        getchar(); /* Clear newline character from input buffer */

```

```

file = fopen(filename, "r");
if (!file) {
    printf("Failed to open the file.\n");
    return;
}

while (i < MAX_LINES && fgets(lines[i], MAX_LENGTH,
file)) {
    lines[i][strcspn(lines[i], "\n")] = 0;
    cleanString(lines[i]);
    i++;
}
fclose(file);
} else {
    while (i < MAX_LINES && !done) {
        printf("Enter a line (or press Enter to finish): ");
        fgets(lines[i], MAX_LENGTH, stdin);
        if (strcmp(lines[i], "\n") == 0) {
            done = 1;
        } else {
            lines[i][strcspn(lines[i], "\n")] = 0;
            cleanString(lines[i]);
            i++;
        }
    }
}
*lineCount = i;
}

```



```

/* Function to output data (to console or file) */
void outputData(char lines[MAX_LINES][MAX_LENGTH], int
lineCount, int toFile, int isOriginal) {
    char filename[MAX_LENGTH];
    FILE *file;
    int i, j;
    char *words[MAX_WORDS];
    int wordCount;
    char *token;

    if (toFile) {
        printf("Enter the filename to write %s text: ", isOriginal ?
"original" : "transformed");
        scanf("%s", filename);
        getchar(); /* Clear newline character from input buffer */

        file = fopen(filename, "w");
        if (!file) {
            printf("Failed to open the file for writing.\n");
            return;
        }

        if (isOriginal) {
            fprintf(file, "Original text:\n");
            for (i = 0; i < lineCount; i++) {
                fprintf(file, "%s\n", lines[i]);
            }
        } else {

```

```

    for (i = 0; i < lineCount; i++) {
        memset(words, 0, sizeof(words));
        wordCount = 0;
        token = strtok(lines[i], " ");

        while (token != NULL) {
            words[wordCount++] = token;
            token = strtok(NULL, " ");
        }

        qsort(words, wordCount, sizeof(char *), compare);

        fprintf(file, "\nTransformed line: ");
        for (j = 0; j < wordCount; j++) {
            fprintf(file, "%s", words[j]);
            if (j < wordCount - 1) {
                fprintf(file, " ");
            }
        }
        fprintf(file, "\n");
    }
}

fclose(file);
} else {
    if (isOriginal) {
        printf("Original text:\n");
        for (i = 0; i < lineCount; i++) {
            printf("%s\n", lines[i]);
        }
    }
}

```

```

    } else {
        for (i = 0; i < lineCount; i++) {
            memset(words, 0, sizeof(words));
            wordCount = 0;
            token = strtok(lines[i], " ");

            while (token != NULL) {
                words[wordCount++] = token;
                token = strtok(NULL, " ");
            }

            qsort(words, wordCount, sizeof(char *), compare);

            printf("Transformed line: ");
            for (j = 0; j < wordCount; j++) {
                printf("%s", words[j]);
                if (j < wordCount - 1) {
                    printf(" ");
                }
            }
            printf("\n");
        }
    }
}

int main() {
    char lines[MAX_LINES][MAX_LENGTH];
    int lineCount;

```

```

int choice;
lineCount = 0;
/* Input menu */
do {
    clearScreen();
    printf("Input Menu:\n");
    printf("1. Input data from console\n");
    printf("2. Input data from file\n");
    printf("Choose an option: ");
    scanf("%d", &choice);
    getchar(); /* Clear newline character from input buffer */

    switch (choice) {
        case 1:
            inputData(lines, &lineCount, 0);
            break;
        case 2:
            inputData(lines, &lineCount, 1);
            break;
        default:
            printf("Invalid choice. Please try again.\n");
    }
} while (choice != 1 && choice != 2);

/* Output menu */
do {
    clearScreen();
    printf("Output Menu:\n");
    printf("1. Output data to console\n");

```

```
printf("2. Output data to file\n");
printf("Choose an option: ");
scanf("%d", &choice);
getchar(); /* Clear newline character from input buffer */

switch (choice) {
    case 1:
        clearScreen();
        outputData(lines, lineCount, 0, 1);
        outputData(lines, lineCount, 0, 0);
        break;
    case 2:
        clearScreen();
        outputData(lines, lineCount, 1, 1);
        outputData(lines, lineCount, 1, 0);
        break;
    default:
        printf("Invalid choice. Please try again.\n");
}
} while (choice != 1 && choice != 2);

return 0;
}
```

Примеры выполнения программы

Пример 1:

```
Input Menu:
1. Input data from console
2. Input data from file
Choose an option: 1
Enter a line (or press Enter to finish): This is the first line.
Enter a line (or press Enter to finish): This is the second line.
Enter a line (or press Enter to finish): This is the third line.
Enter a line (or press Enter to finish): |
```

```
Output Menu:
1. Output data to console
2. Output data to file
Choose an option: 1
```

```
Original text:
This is the first line.
This is the second line.
This is the third line.
Transformed line: This first is line. the
Transformed line: This is line. second the
Transformed line: This is line. the third

Process returned 0 (0x0)   execution time : 138.453 s
Press any key to continue.
```

Пример 2:

```
Input Menu:
1. Input data from console
2. Input data from file
Choose an option: 1
Enter a line (or press Enter to finish): Hello.
Enter a line (or press Enter to finish): Hi.
Enter a line (or press Enter to finish):
```

```
Output Menu:
1. Output data to console
2. Output data to file
Choose an option: 2
```

```
Enter the filename to write original text: Firstline.txt
Enter the filename to write transformed text: Firstline.txt

Process returned 0 (0x0)   execution time : 210.264 s
Press any key to continue.
```

Заключение

Вывод

Во время написания программы для работы с вводом/выводом данных использовались функции из стандартных заголовочных файлов:

1. <stdio.h>:

Функции: printf(), fgets(), scanf(), fopen(), fclose(), fprintf().

Эти функции обеспечивают основной функционал для ввода и вывода данных — как с консоли, так и с файла, а также для записи обработанных данных в файл.

2. <stdlib.h>:

Функции: system() (для очистки экрана), exit() (для завершения программы при ошибках).

Эти функции управляют процессом выполнения программы, обеспечивая работу с внешними командными оболочками и корректное завершение программы в случае ошибок.

3. <string.h>:

Функции: strcspn(), strstr(), strlen(), strcpy().

Эти функции используются для работы с строками — для поиска подстрок, копирования строк и обработки их длины.

4. <ctype.h>:

Функции: tolower() (для преобразования символов в нижний регистр), isalnum() (для проверки, является ли символ буквой или цифрой).

Эти функции предназначены для работы с отдельными символами, например, для их преобразования и проверки.

Поставленная цель была успешно достигнута. Программа корректно решает задачу, включая все этапы: ввод данных (с консоли или из файла), обработка строк, их фильтрация и сортировка. Программа

может выводить результат как на экран, так и сохранять его в файл, в зависимости от выбора пользователя.

Также предусмотрены механизмы для обработки ошибок, например, при невозможности открыть файл или в случае некорректного ввода.

Программа предоставляет пользователю удобный интерфейс для выбора между консольным и файловым выводом, что делает её удобной и гибкой в использовании.